

VivaGuard - Backend Architecture & Endpoint Documentation

1. Executive Summary & Overview

VivaGuard backend is a FastAPI Python web application providing real-time AI technical interview simulations, thesis defense grading, AssemblyAI Voice Agent text-to-audio synthesis, and WebSocket audio streaming. All endpoint handlers are modularized into dedicated controllers with comprehensive docstrings and usage examples.

2. Modularized Backend File Structure & Roles

<code>app/main.py</code>	FastAPI app instance creation, CORS middleware setup, and <code>api_router</code> inclusion.
<code>app/core/config.py</code>	Loads environment variables (.env) such as API keys, host, port, and rate limits via Pydantic.
<code>app/api/deps.py</code>	Dependency container providing <code>SessionBroker</code> , <code>RateLimiter</code> , and <code>AssemblyAIClient</code> singletons.
<code>app/api/v1/api.py</code>	Master API router registering all modular endpoint routers into a unified v1 routing tree.
<code>app/api/v1/endpoints/health.py</code>	Endpoint GET <code>/health</code> to verify backend service uptime and status.
<code>app/api/v1/endpoints/broker.py</code>	Endpoint POST <code>/api/v1/route</code> to broadcast events to active WebSocket subscriber sessions.
<code>app/api/v1/endpoints/debrief.py</code>	Endpoint POST <code>/api/v1/debrief</code> generating STAR framework thesis defense report.
<code>app/api/v1/endpoints/test_generate.py</code>	Endpoint POST <code>/api/v1/test-interview/generate</code> to generate technical practice questions.
<code>app/api/v1/endpoints/test_evaluate_question.py</code>	Endpoint POST <code>/test-interview/evaluate-question</code> to grade candidate text answers & attach audio.
<code>app/api/v1/endpoints/test_evaluate_audio.py</code>	Endpoints POST <code>/test-interview/evaluate-audio</code> & <code>base64</code> to transcribe and evaluate audio blobs.
<code>app/api/v1/endpoints/test_cumulative_report.py</code>	Endpoint POST <code>/test-interview/cumulative-report</code> summarizing final readiness scorecards.
<code>app/api/v1/endpoints/tts.py</code>	Endpoints POST <code>/api/v1/tts</code> and <code>/voice-agent/text-to-audio</code> for speech audio synthesis.
<code>app/api/v1/endpoints/websockets.py</code>	WebSocket handlers <code>/ws/copilot</code> & <code>/ws/audio</code> for live mic streaming & traffic light signals.
<code>app/services/assemblyai_stt.py</code>	AssemblyAI Speech-to-Text transcriber for both recorded audio blobs and streaming mic audio.
<code>app/services/assemblyai_tts.py</code>	AssemblyAI Voice Agent Text-to-Audio generator converting text into WAV speech audio.
<code>app/services/test_simulator.py</code>	LLM-driven question generator, 0-100 scoring engine, and adaptive difficulty tier calculator.
<code>app/services/evaluator.py</code>	STAR framework defense session evaluator analyzing filler words, clarity, and examiner sentiment.
<code>app/services/broker.py</code>	WebSocket session broker managing subscriber connections and broadcasting live evaluation events.

3. API Endpoints Declaration vs Frontend Usage Table

Method	Endpoint Path	Declared In (Backend)	Used In (Frontend)	Purpose
GET	<code>/health</code>	<code>endpoints/health.py</code>	Status Checkers	Health check
POST	<code>/api/v1/route</code>	<code>endpoints/broker.py</code>	Session Broker	Route WS event
POST	<code>/api/v1/debrief</code>	<code>endpoints/debrief.py</code>	<code>DebriefPhase.tsx</code>	STAR defense report
POST	<code>/api/v1/test-interview/generate</code>	<code>endpoints/test_generate.py</code>	<code>TestSimulatorPhase.tsx</code>	Generate questions
POST	<code>/api/v1/test-interview/evaluate-question</code>	<code>endpoints/test_evaluate_question.py</code>	<code>TestSimulatorPhase.tsx</code>	Grade text answer
POST	<code>/api/v1/test-interview/evaluate-audio</code>	<code>endpoints/test_evaluate_audio.py</code>	<code>TestSimulatorPhase.tsx</code>	Transcribe & grade audio
POST	<code>/api/v1/test-interview/cumulative-report</code>	<code>endpoints/test_cumulative_report.py</code>	<code>TestSimulatorPhase.tsx</code>	Cumulative scorecard
POST	<code>/api/v1/tts</code>	<code>endpoints/tts.py</code>	TTS Components	Text-to-Audio synth
WS	<code>/ws/copilot</code>	<code>endpoints/websockets.py</code>	<code>useAudioStreamer.ts</code>	Live copilot stream

4. How to Call Backend Endpoints

A. Generate Practice Questions (cURL):

```
curl -X POST "http://localhost:8000/api/v1/test-interview/generate" \
  -H "Content-Type: application/json" \
  -d '{"domain": "Embedded Systems", "question_count": 2, "difficulty_level": "junior"}'
```

B. Text-to-Audio Voice Agent Synthesis (Python):

```
import requests
res = requests.post("http://localhost:8000/api/v1/tts", json={"text": "Welcome to VivaGuard", "return_json": True})
print("Base64 Audio Length:", len(res.json()["audio_base64"]))
```